Manuel Utilisateur

Projet Génie Logiciel

Groupe 1, gl06

ZIVKOVIC Lucas, SULLET Benjamin, ZOUGGARI Samy,
TRAORÉ Fié Victor, HURSTEL Mathis

20 Janvier 2025

Grenoble INP - Ensimag
Génie Logiciel 2024-2025

Sommaire

Contents

1 Introduction **2**

2 Utilisation **3**
- 2.1 Prérequis . . . 3
- 2.2 Première prise en main . . . 3
- 2.3 Exemple de compilation basique . . . 3
- 2.4 Utilisation des options . . . 4
- 2.5 Utilisation de l'interpréteur `ima` . . . 5

3 Limitations de l'implémentation **6**

4 Messages d'erreur **7**
- 4.1 Erreurs de lexicographie . . . 7
- 4.2 Erreurs de syntaxe hors-contexte . . . 8
- 4.3 Erreurs de syntaxe contextuelle . . . 8
- 4.4 Erreurs d'exécution du code assembleur . . . 9

5 Mode opératoire pour utiliser l'extension **10**
- 5.1 Utilisation des extensions via la ligne de commande . . . 10
- 5.2 Utilité de l'extension **TAB** . . . 11
- 5.3 Limitations de l'extension . . . 11

6 Conclusion **11**

1 Introduction

Bienvenue dans le Manuel Utilisateur de notre compilateur Deca. Ce document a été conçu pour accompagner les utilisateurs dans la prise en main et l'utilisation efficace de notre projet de Génie Logiciel.

Notre compilateur Deca a été développé dans le cadre d'un projet visant à approfondir les compétences en génie logiciel, en programmation et en méthodologies de développement agile. Ce manuel détaille les fonctionnalités, les étapes d'installation, les exemples d'utilisation, ainsi que les limitations de notre implémentation. Le projet Deca a pour objectif principal de fournir un environnement fiable et efficace pour compiler des programmes écrits dans un sous-ensemble du langage Java. À travers ce projet, nous avons exploré des concepts clés comme :

- La création d'un compilateur en Java, couvrant l'analyse lexicale, syntaxique et contextuelle, ainsi que la génération de code assembleur.
- L'adoption de bonnes pratiques de développement logiciel, telles que les tests unitaires, l'intégration continue et le suivi des métriques de qualité logicielle.
- L'utilisation d'outils professionnels comme Maven, ANTLR, Git, et Jacoco, qui nous ont permis d'industrialiser notre approche du développement.

Ce manuel a été structuré pour répondre aux besoins des utilisateurs, qu'ils soient novices ou expérimentés, en proposant une approche progressive et détaillée. Voici les sections principales que vous y trouverez :

- Une présentation pas à pas de la configuration initiale du compilateur sur votre machine.
- Des exemples concrets d'utilisation, depuis la compilation basique jusqu'à l'exécution de programmes Deca.
- Une explication des différentes options de la ligne de commande, permettant d'adapter la compilation selon vos besoins.
- Une description des limitations de l'implémentation actuelle et des pistes pour d'éventuelles améliorations.
- Une analyse des erreurs que vous pourriez rencontrer, avec des conseils pour les résoudre.
- Une prise en main de l'extension `TAB` proposée en parallèle du compilateur.

Ce manuel reflète notre engagement à produire un outil fonctionnel et bien documenté, tout en respectant les contraintes de temps et de complexité imposées par ce projet. Nous espérons qu'il vous permettra d'exploiter pleinement les capacités de notre compilateur et de mieux comprendre le processus de compilation.

Nous vous souhaitons une expérience enrichissante avec notre compilateur Deca. Bonne découverte et bonne utilisation !

2 Utilisation

2.1 Prérequis

Avant toutes choses, pour pouvoir utliser le compilateur du language Deca que nous avons développé, voici quelques prérequis :

- Avoir configurer l'outil `Maven` afin de pouvoir compiler l'entièreté de notre compilateur.
- Avoir installé une machine virtuelle Java pour être en capacité de compiler du code Java.
- Avoir installé la machine abstraite `ima` (pour l'exécution de fichiers assembleur).
- Avoir des notions de bases en terme de compilation.
- Connaître un minimum la syntaxe du language Deca afin de créer vos propres fichiers que vous pourrez compiler.

2.2 Première prise en main

Si vous comptez profiter du compilateur sur un nouvel ordinateur (ex : machine personnelle), veuillez suivre ces quelques instructions afin de le configurer correctement.

Placez-vous à la racine du répertoire où vous avez placé le compilateur et exécutez cette commande. Elle vous permettra de créer toute la structure du compilateur sur votre ordinateur.

```
mvn compile
```

Si vous souhaitez vérifier que tout s'est bien exécuté comme prévu et que votre compilateur est fonctionnel, vous pouvez faire la commande suivante :

```
mvn verify
```

Cela permet d'exécuter la batterie de tests chargés de superviser l'installation du répertoire.

2.3 Exemple de compilation basique

Considérons un exemple simple pour illustrer le processus de compilation :
Supposons que l'on veuille compiler un fichier .deca nommé "mon_fichier.deca" sans utiliser d'option.

Compilation du fichier : Exécutez la commande suivante dans votre terminal :

```
decac mon_fichier.deca
```

Cette commande génère un fichier nommé `mon_fichier.ass`, qui est le code assembleur correspondant à votre fichier Deca. Ce code assembleur est prévu pour être exécuté par la machine virtuelle fournie `ima`. Vous pouvez le retrouver dans le même répertoire que le fichier compilé. À condition d'avoir eu recours à des options de compilation, détaillées plus bas, il est parfaitement normal que la console n'affiche aucun résultat textuel.

Execution du fichier : Exécutez la commande suivante dans votre terminal :

```
ima mon_fichier.ass
```

Cette commande exécute le programme compilé et affiche les résultats attendus dans la console. Si une erreur survient, lors de la compilation ou de l'exécution, veuillez vous reporter à la section Messages d'erreur afin de l'identifier et de la corriger.

2.4 Utilisation des options

Notre compilateur `decac` offre plusieurs options permettant d'adapter le processus de compilation en fonction de vos besoins. Voici une description détaillée de chaque option disponible et comment les utiliser:

- **-b (banner)** : Affiche une bannière contenant notre nom d'équipe. Attention, cette option ne peut être utilisée qu'isolément, sans autre option ni fichier source.

  ```
  decac -b
  ```

- **-p (parse)** : Arrête la compilation après la construction de l'arbre syntaxique et affiche une décompilation du fichier Deca. Cette option est incompatible avec l'option `-v` présentée ci-après.

  ```
  decac -p mon_fichier.deca
  ```

- **-v (verification)** : Arrête la compilation après les vérifications sémantiques, sans produire de fichier de sortie si aucune erreur n'est détectée. Cette option est incompatible avec l'ooption `-p` ci-dessus.

  ```
  decac -v mon_fichier.deca
  ```

- **-n (no check)** : Supprime certains tests d'exécution définis dans la sémantique du langage Deca.

  ```
  decac -n mon_fichier.deca
  ```

- **-r X (registers)** : Limite le nombre de registres banalisés disponibles à `R0` ... `R{X-1}`, où $4 \leq X \leq 16$.

  ```
  decac -r 8 mon_fichier.deca
  ```

- **-d (debug)** : Active les traces de débogage. Répéter cette option augmente le niveau de détail des traces affichées (A noter qu'on ne peut pas excéder trois appels à l'option `-d` pour un même fichier).

  ```
  decac -d mon_fichier.deca
  decac -d -d mon_fichier.deca
  decac -d -d -d mon_fichier.deca
  ```

- **-P (parallel)** : Permet de compiler plusieurs fichiers en parallèle, ce qui peut accélérer le processus de compilation.

  ```
  decac -P fichier1.deca fichier2.deca fichier3.deca
  ```

- **-w (warnings)** : Active l'affichage des messages d'avertissement (*warnings*) pendant la compilation.

```
decac -w mon_fichier.deca
```

Voici quelques exemples d'utilisation combinée. Attention, le recours à plusieurs options simultanées doit respecter cet ordre d'écriture :

```
decac [[-p | -v] [-n] [-r X] [-d]* [-P] [-w] <fichier deca>...]  | [-b]
```

- Pour activer les traces de débogage et afficher les avertissements :

```
decac -d -w mon_fichier.deca
```

- Pour compiler plusieurs fichiers en parallèle et désactiver certains tests à l'exécution :

```
decac -P -n fichier1.deca fichier2.deca
```

- Pour limiter les registres à 8 et activer les avertissements :

```
decac -r 8 -w mon_fichier.deca
```

2.5 Utilisation de l'interpréteur `ima`

L'interpréteur `ima` est un outil permettant d'exécuter des programmes en langage assembleur générés par le compilateur `decac`. Nous détaillons ici son usage et ses différentes options :

`Appel de l'interpréteur :`
Pour utiliser `ima`, exécutez la commande suivante dans un terminal :

ima [options] mon_fichier.ass

Sans option, l'interpréteur assemble et exécute directement le programme spécifié. Cependant, plusieurs options sont disponibles pour ajuster son comportement.

`Options disponibles`

- `-d` : Lance le programme dans un mode interactif appelé *metteur au point*. Ce mode permet de déboguer pas à pas l'exécution du programme.
- `-s` : Exécute le programme directement sans entrer dans le *metteur au point*. Une fois l'exécution terminée (s'il n'y a pas d'erreur), l'interpréteur affiche :
  - Le nombre d'instructions exécutées.
  - Le temps d'exécution total.
- `-r` : Exécute le programme directement sans le *metteur au point*, tout en ajoutant une nouvelle ligne à chaque écriture d'un entier, d'un flottant ou d'une chaîne de caractères.

- `-p` ***nnn*** : Définit la taille de la pile d'exécution en mots (où ***nnn*** est un entier ≥ 0). Par défaut, la pile contient 10 000 mots.
- `-t` ***nnn*** : Définit la taille du tas d'exécution en mots (où ***nnn*** est un entier ≥ 0). Par défaut, le tas contient 10 000 mots.

`Exemples d'utilisation` Voici quelques exemples pour illustrer l'usage de `ima` :

Exécution simple sans option : ima mon_fichier.ass

Exécution avec le *metteur au point* : ima -d mon_fichier.ass

Exécution en mode silencieux avec des statistiques : ima -s mon_fichier.ass

Exécution avec une pile et un tas personnalisés : ima -p 20000 -t 15000 mon_fichier.ass

Exécution avec affichage ligne par ligne : ima -r mon_fichier.ass

En combinant ces options, vous pouvez adapter l'exécution de `ima` selon vos besoins spécifiques.

3 Limitations de l'implémentation

Dans cette section, nous vous présentons les limitations du compilateur, i.e les fonctionnalités que nous n'avons pas réussi à implémenter dans les temps impartis.

Tout d'abord, le compilateur est limité au langage Deca dit "essentiel". C'est-à-dire que le compilateur fonctionne pour la majorité du langage, mais ne supporte pas certaines opérations spécifiques :

- **Cast :** une opération qui convertit une instance d'une classe à une autre sous réserve qu'elles soient bien compatibles. Cependant, le cast est possible et se fait de façon automatique entre un entier et d'un flottant, lorsqu'on assigne ou déclare une variable de type **float** et qu'on lui attribue une valeur **int**, ou bien qu'on réalise une opération entre un **float** et un **int**.
- **instanceof :** de plus, contrairement à Java, le compilateur du langage Deca ne prend pas en charge l'opérateur **instanceof**, il est donc nécessaire d'utiliser des alternatives à cette opération :
  1) Ajouter une méthode comme `isExempleOfType()` dans chaque classe pour vérifier le type d'un objet.
  2) Utiliser un champ identifiant (`type`) dans les classes pour indiquer leur type.
  3) Exploiter le polymorphisme afin d'éviter les vérifications de type explicites et déléguer le comportement aux classes elles-mêmes.

Ces deux limitations sont des améliorations envisageables pour une deuxième version du compilateur, plus poussée. N'étant cependant pas essentielles au bon déroulement du projet, nous n'en avons pas fait une priorité, mais nous pourrions tout à fait nous pencher sur le problème avec plus de temps.

4 Messages d'erreur

Veuillez retrouver ici les différents messages d'erreur que vous serez potentiellement amenés à rencontrer lors de l'utilisation du compilateur.
Ces messages d'erreur sont regroupés en quatre catégories :

- les erreurs de lexicographie,
- les erreurs de syntaxe hors-contexte,
- les erreurs de syntaxe contextuelle,
- les erreurs d'exécution du code assembleur.

`Message d'erreur :` chaque levée d'erreur est accompagnée d'un message de la forme :

"cheminDuTest : ligne : colonne : token recognition error at : symbole qui pose problème"

Chaque type d'erreur est détaillé dans les sous-sections suivantes :

4.1 Erreurs de lexicographie

Les erreurs de lexicographie surviennent lorsqu'un caractère ou une séquence de caractères non reconnue par le langage est détectée (dans ce cas, par le langage Deca). Ces erreurs signalent généralement un problème lié au format ou aux caractères utilisés dans le code.

Analogie : C'est comme si on essayait de glisser des caractères mandarins dans un livre de français : le lecteur ne saurait pas traiter ces caractères puisqu'il ne les reconnait pas.

Ce type d'erreur est souvent lié à une faute de frappe.
Il vous suffit de vous rendre à la ligne du fichier concernée par le problème (visible dans le message levé par la console), de supprimer le caractère non reconnu par le langage et de relancer la compilation.

Ci-dessous 2 exemples de programmes lexicographiquement incorrects :

```
1  {
2      float a = 3.0;
3      float b = 4.8;
4      float €;
5      if (a<b){ max = b; }
6      else { max = a; }
7  }
```

Figure 1: Erreur ligne 4 : € inconnu

```
1  {
2      int x = 5😊;
3      while (x<10){
4          x=x+1;
5      }
6      println(x);
7  }
```

Figure 2: Erreur ligne 2 : emogi inconnu

4.2 Erreurs de syntaxe hors-contexte

Les erreurs de syntaxe hors-contexte apparaissent lorsque le code source ne respecte pas les règles grammaticales définies par la grammaire du langage. Ces erreurs signalent une structure syntaxique invalide, telle qu'une parenthèse manquante ou un mot-clé mal placé. L'utilisation d'instructions et de structures DeCa est soumise à des règles strictes de sémantique, et c'est la violation d'une de ces règles qui déclenche une telle erreur.

Analogie : C'est comme si on inventait de nouveaux mots français, en mettant des lettres bout à bout aléatoirement. Le lecteur essaye de prononcer les mots, sans comprendre ce qu'ils veulent dire.

Ce type d'erreur est souvent lié à une faute de frappe ou un oubli.
Il vous suffit de vous rendre à la ligne et colonne du fichier concernées par le problème (visibles dans le message levé par la console), de supprimer le caractère en trop ou de rajouter celui manquant et de relancer la compilation.

Ci-dessous 2 exemples de programmes syntaxiquement incorrects :



Figure 3: Erreur ligne 2 : "¿" en trop



Figure 4: Erreur ligne 2 : absence d'un ";"

4.3 Erreurs de syntaxe contextuelle

Les erreurs de syntaxe contextuelle se produisent lorsque le code source viole des contraintes de cohérence, telles que l'utilisation de variables non déclarées, des conflits de types ou des appels de méthode incorrects.
Ces erreurs sont les plus dures à détecter car contrairement aux deux précédentes, elles ne sont pas visibles au premier coup d'oeil. Il est nécessaire de se référer aux regles grammaticales DeCa afin de comprendre comment correctement utiliser les instructions qui posent problème.

Analogie : C'est comme si on écrivait des phrases en tirant des mots au hasard du dictionnaire. Chaque mot, individuellement, a un sens, mais l'ensemble de la structure n'est pas cohérente : la phrase ne donne aucune instruction globale mais une personne ne parlant pas français ne peut pas s'en rendre compte.

Ci-dessous 3 exemples de programmes syntaxiquement incorrects :

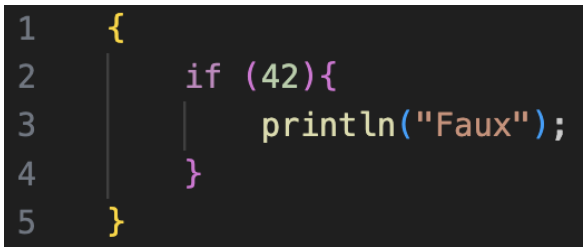


Figure 5: Erreur ligne 2 : if() exige un booléen, pas un int

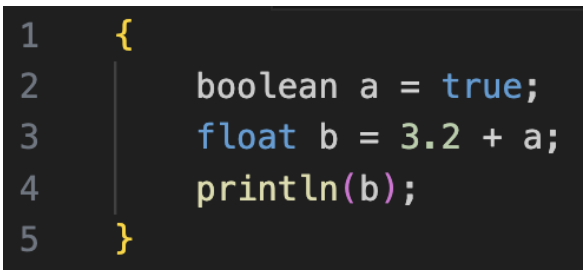


Figure 6: Erreur ligne 3 : addition incompatible entre un float un et booléen

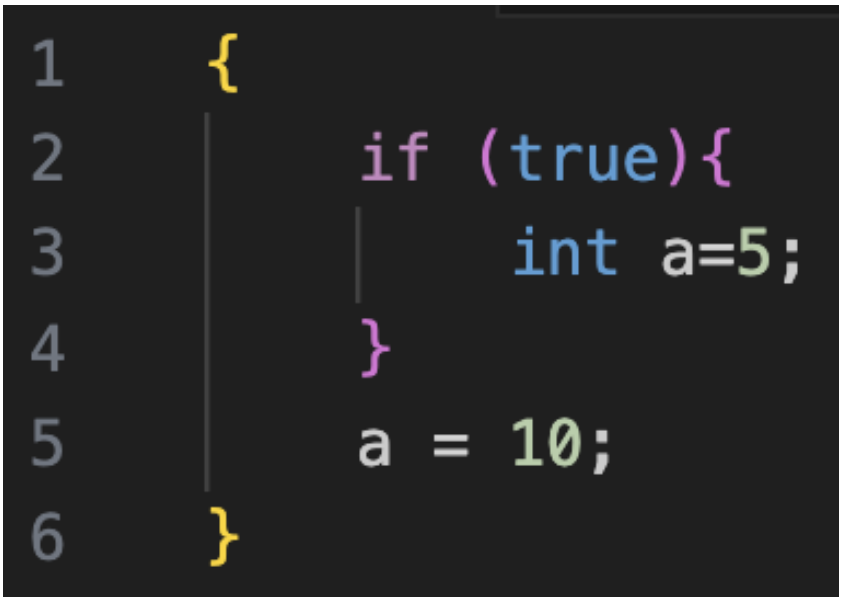


Figure 7: Erreur ligne 5 : "a" n'est connue que dans la boucle

4.4 Erreurs d'exécution du code assembleur

Les erreurs d'exécution du code assembleur surviennent pendant l'exécution du programme généré. Ces erreurs incluent, par exemple, des divisions par zéro, des dépassements de mémoire ou des instructions invalides, souvent causées par des problèmes dans le code source ou au cours de la compilation.

Attention, pour les erreurs d'exécution du code assembleur, le format du message est légèrement différent :

"*Description de l'erreur*"

Ces erreurs sont de loin les plus compliquées à démasquer puisqu'elles ne se produisent qu'au sein de l'ordinateur, en une fraction de seconde. Le programme correspondant est correct mais mal utilisé.
Le meilleur moyen de comprendre le problème est d'utilsier un deboggeur ou d'éxecuter, quand c'est possible, le programme à la main, instruction par instruction.

Ci-dessous 3 exemples de programmes incorrects à l'exécution :

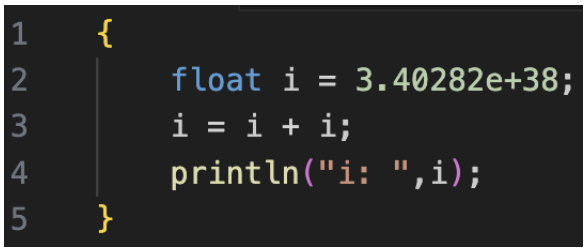


Figure 8: Erreur ligne 3 : débordement float



Figure 9: Erreur ligne 4 : surcharge d'un registre

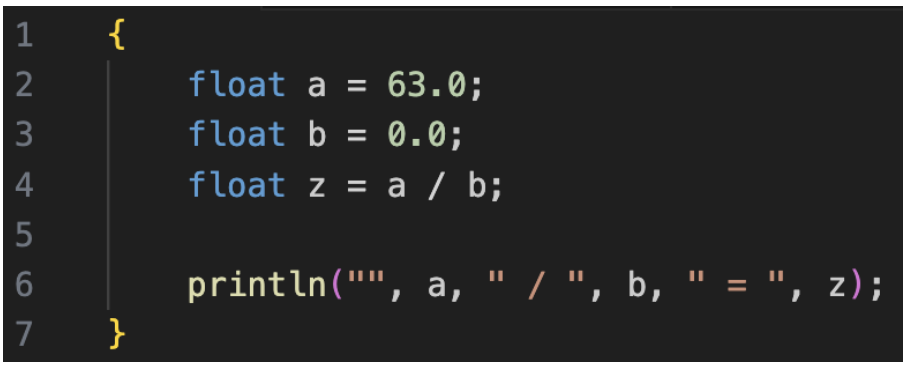


Figure 10: Erreur ligne 4 : division d'un float par 0

5 Mode opératoire pour utiliser l'extension

Il est possible de compiler un langage dit étendu du langage **Deca** classique. L'extension en question est l'extension **TAB** qui permet de manipuler des tableaux statiques. La syntaxe de ces tableaux est similaire à celle des tableaux statiques en **Java**.

5.1 Utilisation des extensions via la ligne de commande

Pour utiliser l'extension, il faut utiliser l'option `-e` via la ligne de commande. Cette option vous permettra de compiler des programmes Deca comportant des tableaux. Par exemple, si dans un fichier nommé "mon_fichier.deca", vous écrivez :

```
{
    int[] x = {1,2,3};
}
```

et que vous le compilez à l'aide de la commande :

```
decac mon_fichier.deca
```

Le terminal affichera une erreur de syntaxe. Cependant, en le compilant avec la commande suivante :

```
decac -e mon_fichier.deca
```

Cela ne posera aucun problème puisque ce programme sera valide du point de vue du Deca "étendu".

5.2 Utilité de l'extension TAB

L'extension **TAB** enrichit le langage **Deca** en introduisant la manipulation de tableaux statiques, ce qui ouvre de nouvelles possibilités dans la conception et l'écriture de programmes complexes. Ces fonctionnalités permettent de gérer efficacement des structures de données indexées, essentielles dans de nombreux domaines :

- **Simplicité et expressivité** : Grâce à une syntaxe familière, similaire à celle de **Java**, les tableaux statiques offrent une approche intuitive pour stocker et manipuler des collections d'éléments.
- **Performance** : La gestion statique des tableaux garantit une allocation mémoire prédictible, idéale pour les applications nécessitant un contrôle strict des ressources.
- **Flexibilité** : Cette extension permet de développer des algorithmes plus avancés, tels que des tri, des recherches ou des calculs mathématiques impliquant des ensembles de données.

Les tableaux statiques introduits par l'extension **TAB** ouvrent la voie à plusieurs applications et prolongements intéressants comme les calculs matriciels, la gestion de données et l'optimisation de simulations.

5.3 Limitations de l'extension

L'extension **TAB** n'a pas de limitations significatives au niveau syntaxique ou contextuel. Pour la réaliser, nous nous sommes contentés de suivre et de copier la syntaxe utilisée en Java pour les tableaux statiques.

Cependant, certaines limitations peuvent survenir dans le cadre de la réalisation de la bibliothèque de calcul matriciel.
Cette bibliothèque peut être utilisée en ajoutant la ligne suivante au début du fichier Deca à compiler :

```
#import Tab.decah
```

Cependant, cette bibliothèque n'est pas exhaustive = nous ne pouvons pas répertorier toutes les opérations possibles avec des tableaux statiques.
Pour connaître les méthodes implémentées dans la bibliothèque, nous vous invitons à consulter le fichier `Tab.decah`. Si vous souhaitez obtenir plus d'informations concernant la bibliothèque, vous pouvez également consulter la documentation spécifique à l'extension. Une partie de cette documentation est entièrement dédiée à la conception de la bibliothèque de calcul matriciel.

6 Conclusion

Nous espérons que ce guide vous a permis de comprendre pleinement comment utiliser notre compilateur Deca. Nous avons réalisé cet outil pour répondre aux besoins de notre projet de Génie Logiciel mais celui-ci pourrait tout à fait être retravaillé pour offrir de nouvelles fonctionnalités.

Nous vous encourageons à le tester sur vos propres programmes Deca afin de bien cerner toutes les fonctionnalités offertes. Si vous souhaitez avoir des informations plus techniques sur la conception architecturale, veuillez vous référer à la documentation prévue à cet effet. Enfin en cas de questions vous pouvez nous contacter via nos adresses e-mails académiques :

- Mathis.Hurstel@grenoble-inp.org
- Fie-Victor.Traore@grenoble-inp.org
- Lucas.Zivkovic@grenoble-inp.org
- Samy.Zouggari@grenoble-inp.org
- Benjamin.Sullet@grenoble-inp.org